

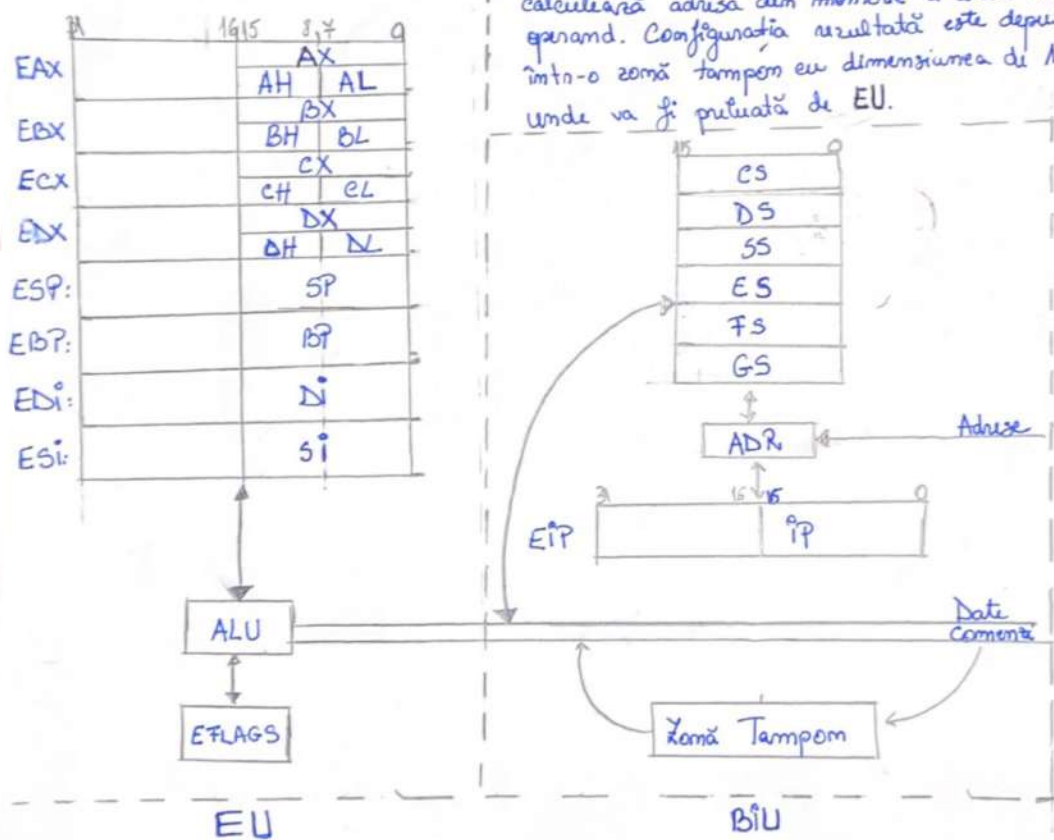
Valoarea în zecimal	Valoarea în hexazecimal	Nr. bimar compusă cifră hexa
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Arhitectura microprocesorului X86 (IA-32)

Structura microprocesorului

2 componente principale

- EU (Executive Unit) - execută instrucțiunile mașină prin intermediul componentei ALU (Arithmetic and Logic Unit)
- BIU (Bus Interface Unit) - pregătește executia fiecărei instrucțiuni mașină. Citește o instrucțiune din memorie și calculează adresa din memorie a unui eventual operand. Configurația rezultată este depusă într-o zonă temporară cu dimensiunea de 15 octeți, unde va fi preluată de EU.



- EU și BIU lucrează în paralel, în timp ce EU execută instrucțiunea curentă, BIU pregătește instrucțiunea următoare. Cele două acțiuni sunt sincronizate, ele care termină prima așteaptă după cealaltă.
- toate comenzile ajung în ALU, unde date ajung în ALU

ALU: $\left\{ \begin{array}{l} \text{aritmetică (+, -, \cdot, :)} \\ \text{luarea deciziilor (if-uri), comparare} \\ \text{operații logice: și, sau, not (pe biți)} \end{array} \right.$

↓

element crucial în arhitectura unui microprocesor

CPU: Central Processing Unit
 = combinare ALU și registre,
 unitatea de comandă și
 control și interconexiunile

RAM: (Random Access Memory)

= tip de memorie în care un calculator poate accesa datele în mod aleatoriu

- caracteristici:

- **viteză de acces rapid**: este mai rapid decât memoria de stocare permanentă; acces-ul la date în RAM este rapid pt că informațiile sunt stocate electronic și pot fi citite/scrise rapid.
- **volatilitate**: datele din RAM sunt volatile (= sunt șterse când calculatorul este oprit/repornit) RAM-ul necesită alimentare constantă pt a menține datele stocate.
- **capacitate de lucru**: este folosită de CPU pt a executa programe și pt a procesa date în timp real. Cu cât un calculator are mai mult RAM cu atât poate gestiona o cantitate mai mare de informații și sarcini simultan, fără a reduce performanța.

Memoria de stocare permanentă: - datele rămân intacte chiar și când sursa de alimentare a dispozitivului este oprită.

- Hard Disk Drive (HDD)
- Solid State Drive (SSD): utilizează memoria flash
- Memorie Flash: utilizată în SSD, stick-uri USB, carduri de memorie
- CD, DVD, Blue-Ray
- Cloud Storage: stochează datele pe servere remote, accesibile prin internet

Registru unui microprocesor = capacități de memorare foarte mici (8/16/32 biți), dar foarte rapide ca viteză de acces și ei sunt utilizați pentru stocarea temporară a operanzilor, instrucțiunilor (cu care operează în momentul acela procesorul).

Aste operanzi pot fi $\left\{ \begin{array}{l} \text{adrese} \\ \text{comenzi} \\ \text{date} \end{array} \right.$ | biți din pe căi diferite

Procesorul lucrează mai ales cu numere întregi, sarcinile cu numere reale îi revin unui coprocesor matematic (device special)

Bit-ul este unitatea primară de reprezentare a informației. $\begin{matrix} 0 \\ \swarrow \\ 1 \end{matrix}$

! Nu se poate accesa un singur bit.

Octetul (byte) este cea mai mică unitate de memorie adresabilă.

este cea mai mică unitate adresabilă la nivelul memoriei. Avem nevoie de adresă pt accesarea octetului.
cea mai mică unitate de informație adresabilă și manipulabilă

! Octetul are adresă, bitul nu are.

Registrii generali EU

↳ pot fi folosiți cum dorim, dar au anumite contexte predefinite (immutabile, etc)

Registru = capacități de memorie de la nivelul procesorului (în nucleul procesorului), de dimensiune mică de memorie (8/16/32/64 bit), dar foarte rapide ca viteză de acces la informație. Au rolul de a stoca temporar operații cu care lucrează un procesor în momentul curent.

Operații pot fi: date, coduri de comenzi, adrese.

EAX - Registru acumulator

- cel mai utilizat registru din procesor,
- este folosit de majoritatea instrucțiunilor procesorului drept unul dintre operații

EBX - Base Register / Registru general

ECX - Counter Register / Registru contor

- pt instrucțiunile care au nevoie de indicații numerice.
- pt instrucțiuni în principal de citare indicând nr de pași ce trebuie efectuați

EDX - Data Register

- împreună cu EAX se folosește în calculele ale căror rezultate depășesc un dword/32

Registrii **ESP** și **EBP** sunt regiștri destinați lucrului cu stivă.

Stivă = o zonă de memorie în care se pot depune succesiv valori, extragerea lor făcându-se după principiul **LIFO**.

LIFO: last in, first out

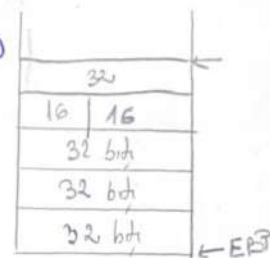
ESP: Stack Pointer

- punctează spre elementul **ultim** introdus în stivă (vârful stivei)

EBP: Base Pointer

- punctează spre **primul** element introdus în stivă (baza stivei)

SS = Stack Segment (segment de stivă)



De ce este importantă stiva pentru procesor?

1. Gestionarea Apelurilor de Funcție

Atunci când o funcție este apelată într-un program, adresa de întoarcere și alte informații necesare pentru a reveni la locul corect din program după finalizarea execuției funcției respective sunt adăugate în stivă.

Acest lucru îi permite procesorului să revină la execuția corectă a programului după finalizarea unei funcții.

2. Gestionarea Parametrelor Funcțiilor

Valoarea parametrilor sunt de asemenea stocate pe stivă și sunt apoi accesate de funcție și folosite.

3. Stocarea variabilelor locale

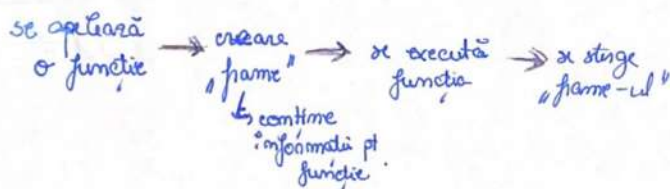
Valoarea locale ale unei funcții sunt de obicei stocate în stivă.

Principiul factorizării
= descompunerea unui sistem complex sau a unei entități în componente.
→ folosim funcții

Procedurile (funcțiile) se execută în ordinea stivei.

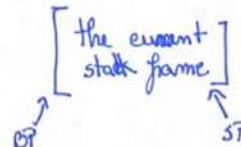
Stiva de execuție

Ordinea de activare/dezactivare a subrutinelor în orice program respectă disciplina LIFO. Spațiul de execuție a oricărui program fiind așa numita stivă de execuție (The Run Time Stack). Aceasta este nativitatea pentru care procesorul alocă 3 registre pentru stivă și 0 pt coadă.



Memorie = secvență de bytes

Memoria este organizată pe segmente.



Registrii ESI și EDI sunt registre de index utilizate de obicei pentru accesarea elementelor din șiruri de caractere sau de cuvinte.

EDI - Destination Index

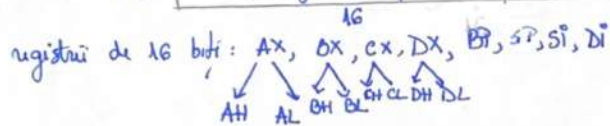
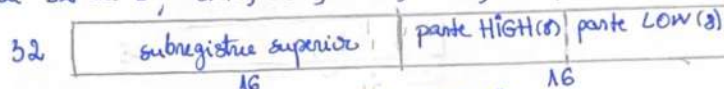
ESI - Source Index

Oricare program scris de mine nu a fost altceva decât o succenta de instrucțiuni de atribuire a cărei ordine a fost dictată de către miștile instrucțiunilor de control din cadrul limbajului respectiv (if, while, etc)

! **EDI** și **ESI** au fost purăeți la nivelul procesorului x86 datorită necesității lucrului cu siruri de elemente.

Arhitectura și limbajul nu înțeleg noțiunea de tablou (array). Sirurile trebuie simulate de către programator prin gestionarea unei baze (începutul string-ului) și al unui index. Indecșii pot fi simulați și gestionați prin registre ESI și EDI.

registrii de 32 de biți: **AX, EBX, ECX, EDX, EBP, ESP, ESI, EDI**



Flagurii

flag = indicator pe un bit

UOE - ultima operație efectuată

O configurație a registrului de flaguri indică un anumit simț al execuției fiecărei instrucțiuni.

EFLAGS (Status Register) - 32 de biți, dar se folosesc uzual 9

31	30	...	12	11	10	9	8	7	6	5	4	3	2	1	0
X	X	...	X	OF	DF	IF	TF	SF	ZF	X	AF	X	OF	X	CF

CF (Carry Flag) - flag-ul de transport

CF = $\begin{cases} 1 & \text{dacă în cadrul UOE s-a efectuat transport} \\ 0 & \text{dacă nu a fost transport} \end{cases}$

$$\begin{array}{r} 1001\ 0011 + \\ 0111\ 0011 \\ \hline 01000\ 0110 \end{array}$$

Carry Flag simulează depășirea în cazul interpretării FĂRĂ SEMN

OF (Overflow Flag) - pt depășire CU SEMN

OF = $\begin{cases} 1 & \text{dacă rezultatul UOE NU a încăput în spațiul rezervat operandilor în interpretarea cu SEMN} \\ 0, \text{ înveș} \end{cases}$

OF = 0

! Pentru aceeași adunare există două interpretări SIMULTANE astfel că sunt necesare 2 flaguri

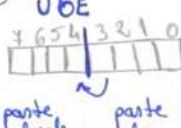
PF (Parity Flag) - valoarea lui se stabilește a7. împăună cu nr de biți 1 din octetul cel mai puțin semnificativ al reprezentării rezultatului UOE să rezulte un nr impar de cifre 1.

rezultat $\begin{cases} 1 & \text{paritate pară} \\ 0 & \text{paritate impară} \end{cases}$

mov eax, 1 ; PF = 0
add eax, 1 ; PF = 1

1001 0011
0111 0011
① 0000 0110
CF = 0
OF = 0
PF = 1
AF = 0

AF (Auxiliary Flag) - indică valoarea transportului de la bitul 3 la bitul 4 al rezultatului UOE

rezultat: 

ZF (Zero Flag)

ZF = $\begin{cases} 0 & \text{dacă rezultatul diferă de 0} \\ 1 & \text{dacă rezultatul} == 0 \end{cases}$

SF (Sign Flag) = flag-ul de semn (se aștează la semnul UOE)

SF $\begin{cases} 1 & \text{dacă rezultatul UOE este un număr strict negativ} \\ 0 & \text{în caz contrar.} \end{cases}$

TF (Trap Flag) - flag de depanare, nu avem acces direct la el
- dacă are valoarea 1, atunci mașina se oprește după fiecare instrucțiune.

IF (Interrupt Flag) - flag de întrerupere
- se denumesc prin intermediul lui secțiuni critice

cli ; IF = 0 $\rightarrow$ pt 16 biți
secțiune critică $\left\{ \begin{array}{l} \text{mov eax, 2} \\ \dots \\ \text{add ax, bx} \end{array} \right.$
sti ; IF = 1

DF (Direction Flag) - pt operare asupra sirurilor de octeți sau cuvinte
- se activează înainte de execuție

DF $\begin{cases} = 0 & \text{parcursere ascendentă ST} \rightarrow \text{DR} \\ = 1 & \text{parcursere descendentă DR} \rightarrow \text{ST} \end{cases}$

Categorii de flag-uri

- 1) stăte ea urmărirea a acțiunii UOE: CF, PF, AF, ZF, SF, OF
(ce s-a întâmplat)
- 2) stăte de programator anterior execuției unei instrucțiuni: (ce se va întâmpla)
 CF, TF, DF, IF

! CF singurul flag care apare în ambele categorii

Instrucțiuni specifice de setare a valorilor flag-urilor. (!)

= instrucțiuni de acces direct la 3 flag-uri.

DF $\left\{ \begin{array}{l} CLD: DF=0 \\ STD: DF=1 \end{array} \right.$

CF $\left\{ \begin{array}{l} CLE: CF=0 \\ STC: CF=1 \\ CMC: \text{complement } CF \end{array} \right.$

IF $\left\{ \begin{array}{l} CLI: IF=0 \\ STI: IF=1 \end{array} \right.$
16 biți

! nu există instrucțiuni de acces la TF .

Registri de adresă și calculul de adresă

Adresa unei locații = no de octeți consecutivi dintre începutul memoriei RAM și începutul locației respective

Segment = o succesiune continuă de locații de memorie, menite să deservesc scopuri similare în timpul execuției unui program (categoria de informației) - o diviziune logică a memoriei unui program: caracterizată prin

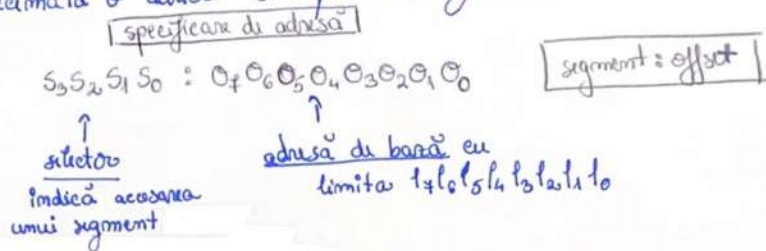
- adresă de bază (început): 32 biți
- limită (dimensiune): 32 biți
- tip.

Offset = Deplasament = adresa unei locații față de începutul unui segment
= numărul de octeți aflați între începutul segmentului și locația în cauză.
un offset este valid $\Leftrightarrow$ valoarea sa numerică (pe 32 de biți) nu depășește limita (dimensiunea) segmentului la care se referă

Specifierea de adresă = Adresă logică = pachet formată dintr-un selector de segment și un offset

- selector de segment = valoare numerică pe 16 biți care identifică (indicează/selecționează) în mod unic segmentul accesat și caracteristicile acestuia
! este definit și furnizat de către sistemul de operare.

În scriere hexazecimală o adresă se exprimă sub forma:



condiție: $O_7 O_6 O_5 O_4 O_3 O_2 O_1 O_0 \leq 14 13 12 11 10 9 8 7$
 $\uparrow \qquad \qquad \qquad \uparrow$
 offset căutat limita segmentului

! Baza și limita sunt determinate de către procesor în urma aplicării mecanismului de segmentare.

Determinarea adresei de segmentare din specificarea de adresă se face printr-un calcul de adresă cf. formulei:

$$a_7 a_6 a_5 a_4 a_3 a_2 a_1 a_0 := b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0 + \overbrace{O_7 O_6 O_5 O_4 O_3 O_2 O_1 O_0}^{\text{adresa calculată (h)}}$$

adresa limită / adresă de segmentare

ADRESĂ FAR: specificare de adresă
ADRESĂ NEAR: doar offset

sub 16 biți, adresa de segment era adresa fizică a începutului resp. segment

exemplu: 8 : 1000h

Pt a calcula adresa limită ei-i corespunde acelei specificații, procesorul va:

- 1) verifică dacă segmentul corespunde valorii de selectare 8 a fost definit de către SO și a blocarea accesul dacă nu a fost definit un offset de segment
- 2) extrage adresa de bază (B) și limita acestui segment (L)
 * B = 2000h, L = 4000h (este o opțiune la ale cărei detalii nu avem acces, ea derivându-se exclusiv între procesor și SO)
- 3) verifică dacă offset-ul depășește limita segmentului
 1000h > 4000h?

(în caz de depășire accesul ar fi blocat: memory violation error)

- 4) adună offset-ul cu B, obținând în cazul nostru adresa limită 2000h (1000h + 2000h). Acest calcul este efectuat de către componenta ADP din BIU

SO		
	Adresa de bază	Size of
mode 3	0F302h	h
ordine 8	AD14h	h
11		
descripți de segment 14	...	

↳ segmente

Segmentare = modelul de adresare segmentată (exemplu)

Caracteristică: segmentele încep de la 0, au dimensiunea maximă posibilă 4 GiB, orice offset este automat valid $\rightarrow$ segmentarea nu contribuie în calculul adreselor.

$\Rightarrow b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0 = 00000000$, calculul pt adresare logică segment:offset
 se rezulta: $a_7 a_6 a_5 a_4 a_3 a_2 a_1 a_0 = 00000000 + 0_4 0_6 0_5 0_4 0_3 0_2 0_1 0_0$

$$\begin{array}{c} \Downarrow \\ \underbrace{a_7 a_6 a_5 a_4 a_3 a_2 a_1 a_0}_{\text{adresă}} = \underbrace{0_4 0_6 0_5 0_4 0_3 0_2 0_1 0_0}_{\text{offset}} \end{array}$$

Model de memorie flat = adrese de memorie sunt exprimate printr-o singură dimensiune și sunt continue
 (majoritatea SO)
 - 0 = începutul memoriei
 = un singur segment

Paginare - specifică x86

= mecanism de control al accesului la memorie, independent de adresarea segmentată

- implică împărțirea memoriei virtuale în **pagini**, care sunt asociate (traduse) memoriei fizice disponibile

$$1 \text{ page} = 4 \text{ KiB} = 2^{12} \text{ bytes} = 4096 \text{ bytes}$$

Configurarea și controlul mecanismelor de segmentare și paginare sunt sarcini SO-ului. Segmentarea intervine în specificarea de adrese, paginarea este complet transparentă din perspectivă programator de utilizator.

Modul de execuție al procesorului $\xrightarrow{\text{implementare}}$ $\left\{ \begin{array}{l} \text{calculul de adrese} \\ \text{folosirea mecanismelor de segmentare și paginare} \end{array} \right.$

Moduri de execuție x86:

- mod real, pe 16 biți folosind cursorul de memorie de 16 biți, memoria limitată 1MB
- ! - mod protejat pe 16 sau 32 biți, caracterizat prin folosirea paginării și segmentării
- mod virtual 8086, permite rularea programelor de tip mod real alături de cele de mod protejat
- long mode, 64 sau 32 biți, paginarea obligatorie, segmentarea dezactivată

Arhitectura x86 : 4 tipuri de segmente

- segment de cod, conține instrucțiuni maxime
- segment de date, conține date asupra cărora se acționează cu instr.
- segment de stivă
- extra segment (segment suplimentar de date)

! fiecare program este format din unul sau mai multe segmente.

Registru (16 biți, în BIU) / Registru de segment

- | | | | | |
|---------------------------|---|----|---|--|
| conțin
valoare | { | CS | { | - au semnificație prestabilită |
| | | DS | | - determină adresele de început și dimensiunile segmentelor active |
| selectoare
segm active | { | SS | { | |
| | | ES | | |
| | | FS | | - nu au valori predefinite |
| | | GS | | - pot deține selectori indicând către segmente suplimentare |

Registru EIP(32) / IP(16) conține offset-ul instrucțiunii curente în cadrul segm de cod curent, fiind exclusiv manipulat de BIU

Notiuni asociate adresării

Notiune	Reprezentare	Descriere
Specificare de adresă Adresă logică Adresă FAR	selector(16): offset(32)	Defineste complet atât segmentul cât și displacementul în cadrul acestuia
Selectore de segment	16 biți	Identifică unul dintre segmentele disponibile. Ca valoare numerică acesta codifică poziția descriptorului de segment selectat în cadrul unei table de descriptori.
Adresă liniară (adresă de segment)	32 biți	Început segment + offset, reprezintă rezultatul calculului de adresă.
Offset Adresă NEAR	offset(32)	Defineste doar componenta de offset (considerând segmentul cunoscut ori folosind modelul de memorie flat)
Adresă fizică efectivă	cel puțin 32 biți	Rezultatul final al segmentării plus, eventual, prelucrării. Adresa finală determinată de către BIU, indicând în memorie fizică (hardware)

Adrese FAR și NEAR

- pentru a adresa o locație din memoria RAM sunt necesare două valori:
 - una indică segmentul
 - una indică offset-ul
 - pt a simplifica referința la memorie, microprocesorul derivă, în lipsa unei alte specificații, adresa segmentului din unul dintre registrele de segment CS, DS, SS sau ES. Alegerea implicită a unui registru de segment se face după niște reguli proprii instrucțiunii folosite.
- Adresa NEAR = adresă în care se specifică doar offset-ul, segmentul va fi preluat implicit dintr-un registru de segment
- o adresă near se află mereu în interiorul unuia din cele patru segmente active

Adresa FAR = adresă în care programatorul indică explicit un selector de segment
= o specificație COMPLETĂ de adresă:

- 1) SS, DS, ES, FS, GS : specificare offset ; SS, DS, ES, FS, GS constantă
- 2) registru segment: specificare offset ; registru segment: CS, DS, SS, ES, FS, GS
- 3) FAR [variabilă], variabilă de tip word și conține 6 octeți constituind adresa FAR (cea ce numim variabilă pointer în limbajele de nivel înalt)

ex) `jump FAR ...` → sare în alt segment

Un word nu se poate manipula sau accesa ca entitate

Formatul intern al adresei FAR:

- la adresa mai mică se află offset-ul, iar la adresa mai mare cu 4 (word-ul care urmează după word-ul curent) se află word-ul ce conține selectorul care indică segmentul.
- reprezentarea adreselor respectă little-endian: partea cea mai puțin semnificativă are adresa cea mai mică, partea cea mai semnificativă are adresa cea mai mare.

Aritmetica de pointeri

- în cadrul sistemului de adresare se fac operații cu pointeri

Aritmetica de pointeri/adrese = utilizarea de expresii aritmetice, care au ca operanzi adrese

Operații (3)

- scăderea a două adrese = nr de octeți dintre două adrese de memorie

⇓
valoare scalară

$$\text{adresă} - \text{adresă} = \text{nr}$$

$$\text{ex } 9 - 7$$

(în C = sizeof(array) sau nr de elemente)

- adunarea unei constante numerice la o adresă

$$\text{adresă} + \text{constantă numerică} = \text{adresă}$$

(identificarea unui elem prin
indexare - a[i])

$$\text{ex } 9 + 9$$

- scăderea unei constante numerice la o adresă

$$\text{adresă} - \text{constantă numerică} = \text{adresă}$$

$$\text{ex } 9 - 7$$

! adresă = pointer

pt că returnează
un pointer sunt
utile pt referirea de
elemente dintr-un
array/zonă de mem.

! adunarea a doi pointeri nu e permisă

Exemplu curs 5

Valoare stânga VS valoare dreaptă a unei atribuiri

L-value

R-value

atribuire: $i := i + 1$ (adresa lui $i \leftarrow$ valoarea lui $i + 1$)

LHS(i) / L-value
= adresă

RHS(i) / R-value
= conținut

Operații fără sens

- adunarea a doi pointeri
- immutabile
- împărțire

Excepție

Adunarea de valori de
registru (NU de pointeri)
este permisă la formele
de calcul a offsetului unui
operand.

Dezincrințirea = extragerea unei valori de la o adresă

- implicită în 99% din limbaje
- operația de dezincrințire [...]]

Q De ce orice instrucțiune sub
arhitectura x86 are maxim 2
operanzi.?

? De ce înmulțirea a provocat
spațiu pt rezultat dublu cât
cel al operandilor (asigurându-se
că rezultatul va încăpea în
spațiul rezervat, iar pt adunare și
scădere nu s-a provocat un spațiu
dif pt rezultatul final.?

CURS 3

1

CAPITOLUL 3

ELEMENTELE DE BAZA ALE LIMBAJULUI DE ASAMBLARE

Limbaajul mașină al unui sistem de calcul (SC) este format din totalitatea instrucțiunilor mașină puse la dispoziție de procesorul SC. Acestea se reprezintă sub forma unor șiruri de biți cu semnificație prestabilită.

Limbaajul de asamblare al unui calculator este un limbaj de programare în care setul de bază al instrucțiunilor coincide cu operațiile mașinii și ale cărui structuri de date coincid cu structurile primare de date ale mașinii. **Limbaaj simbolic. Simboluri - Mnemonice + etichete.**

Elementele cu care lucrează un asamblor sunt:

- * **etichete** - nume scrise de utilizator, cu ajutorul cărora se pot referi date sau zone de memorie.
- * **instrucțiuni** - scrise sub forma unor mnemonice care sugerează acțiunea. Asamblorul generează octeții care codifică instrucțiunea respectivă.
- * **directive** - sunt indicații date asamblorului în scopul generării corecte a octetilor. Ex: relații între modulele obiect, definirea unor segmente, indicații de asamblare condiționată, directive de generare a datelor.
- * **contor de locații** - număr întreg gestionat de asamblor. În fiecare moment, valoarea contorului coincide cu numărul de octeți generați corespunzător instrucțiunilor și directivelor deja întâlnite în cadrul segmentului respectiv (deplasamentul curent în cadrul segmentului). Programatorul poate utiliza această valoare (accesare doar în citire!) prin simbolul '\$'. **Fiecare segment are propriul său contor de locații !!!**

NASM supports two special tokens in expressions, allowing calculations to involve the current assembly position: the \$ and \$\$ tokens. \$ evaluates to the assembly position at the beginning of the line containing the expression; so you can code an infinite loop using JMP \$.

\$\$ evaluates to the start of the current section; so you can tell how far into the section are by using (\$-\$).

Directiva SECTION

```
section .data
    db 'hello'
    db 'h', 'e', 'l', 'l', 'o'
    data_segment_size equ $-$
```

data-segment-size = 10

\$ - here
 \$\$ - start of segment
 \$ > \$\$

\$-\$ = the distance from the beginning of the segment AS A SCALAR (constant numerical value)!!!!!!

\$ - is an offset = POINTER TYPE !!!! It is an address !!!!

\$\$ - is an offset = POINTER TYPE !!!! It is an address !!!!

\$ means "address of here".

\$\$ means "address of start of current section".

So \$-\$ means "current size of section".

For the example above, this will be 10, as there are 10 bytes of data given.

Dacă nu am definite secțiuni explicite, atunci \$\$=inceputul segmentului respectiv

aici - altundeva ← NU merge
 altundeva - aici ← merge
 etichetă - etichetă ← merge

\$ - here
 \$\$ - start segment

Curs 6

Formula de determinare a offset-ului: (16 biți)

$$\underbrace{[BX/BP]}_{\text{baza}} + \underbrace{[SI/DI]}_{\text{index}} + [\text{constantă}]$$

Asamblen = program translator, compilator al limbajului assembly

= generează octeți corespunzători directivelor și instrucțiunilor la nivelul codului sursă

Compilator = analizează corectitudinea sintactică a codului sursă

Contor de locații

Segment data

```
a db 17, -2, 0ffh, 'xyz', ...
db ....
db ....
```

Contorul de locații

= nu este atribuită (VALUE)

= câți octeți sunt generați de la începutul segmentului

;lga db \$-a (mov [lga],...); ok //aritmetica de pointeri – scăderea a 2 pointeri = scalar (constanta numerică) - lga=variabila de memorie (mov [lga],...)

;lga dw \$-SS ; Corect numai DACA a este primul element definit în data segment !!!!

;lga EQU \$-a ; ok ! însa mov [lga],... este syntax error !!! pt ca lga NU este o variabilă alocată... ci o constantă! (fără alocare de memorie)

EQU - constantă

;lga dw \$-data ; corect in TASM/MASM, INCORECT in NASM sub 32 biți !!!
syntax error – "Expression is not simple or relocatable"

;lga dw lga-a !!!!!!!!

b EQU 27 ; b NU este un offset !!!!
c dd 12345678h

BYTE: 1 octet
WORD: 2 octeți
DWORD: 4 octeți
QWORD: 8 octeți

;lga dw b-a ; syntax error !!!!! b is NOT an address !!!

;lga dw c-a ; ok !!!!

lga dw \$-4 ; ok !!!

lg dw \$-a ; length (a) + 4 !!!

Dacă nu se utilizează nici o directivă section în mod explicit, simbolul \$ se va evalua implicit la offset-ul începutului de segment.

Elementul sintactic ":" se pune obligatoriu când definim etichete de cod (ex: "start:") însă nu trebuie pus dacă definim o etichetă de date (ex: definirea de variabile "a db 17")

3.1. FORMATUL UNEI LINII SURSĂ

Formatul unei linii sursă în limbajul de asamblare x86 este următorul:

[*etichetă*[:]] [*prefixe*] [*mnemonică*] [*operandi*] [*comentariu*]

Ilustrăm conceptul prin intermediul a câteva exemple de linii sursă:

```
aici: jmp acolo           ; avem etichetă + mnemonică + operand + comentariu
      repz cmpsd          ; prefix + mnemonică + comentariu
      start:              ; etichetă + comentariu
                        ; doar un comentariu (care putea lipsi și el)
      a dw 19872, 42h     ; etichetă + mnemonică + 2 operandi + comentariu
```

Caracterele din care poate fi constituită o *etichetă* sunt următoarele:

- Litere, atât A-Z cât și a-z;
- Cifre de la 0 la 9;
- Caracterele `_`, `$`, `$$`, `#`, `@`, `.`, `~`, `.` și `?`

Ca și prim caracter al unei etichete sunt permise doar litere, `_` și `?`

Aceste reguli sunt valabile pentru toți *identificatorii* valizi (denumiri simbolice, precum nume de variabile, etichete, macro, etc).

Identificatorii NASM sunt *case sensitive*, limbajul diferențiind literele mari de cele mici privitor la denumirile utilizator. Aceasta înseamnă că un identificator `Abc` este diferit de identificatorul `abc`. Pentru denumirile implicite parte a limbajului, cum ar fi cuvintele cheie, mnemonicile și numele regiștrilor, nu se diferențiază literele mari de cele mici (acestea sunt *case insensitive*).

La nivelul limbajului de asamblare se întâlnesc două categorii de etichete:

- 1). **etichete de cod**, care apar în cadrul secvențelor de instrucțiuni cu scopul de a defini destinațiile de transfer ale controlului în cadrul unui program. **Pot apărea și în segmente de date !**
- 2). **etichete de date**, care identifică simbolic unele locații de memorie, din punct de vedere semantic ele fiind echivalentul noțiunii de *variabilă* din alte limbaje. **Pot apărea și în segmente de cod !**

Valoarea unei etichete în limbaj de asamblare este un număr întreg reprezentând adresa instrucțiunii, directivei sau datelor ce urmează etichetei.

Distincția dintre referirea adresei unei variabile sau a conținutului asociat acesteia în NASM se face după regulile:

- Când este specificat **între paranteze drepte, numele variabilei desemnează valoarea variabilei**, de exemplu `[p]` specifică accesarea valorii variabilei `p`, similar cu modul în care `*p` semnifică dereferențierea unui pointer (accesul la conținutul indicat prin valoarea pointerului) în C;
- În orice alt context **numele variabilei reprezintă adresa variabilei**, spre exemplu, `p` este întotdeauna adresa variabilei `p`;

Exemple:

```
✓ mov EAX, et      ; încarcă în registrul EAX adresa datelor sau a codului marcat cu eticheta et (4 octeți)
0 mov EAX, [et]    ; încarcă în registrul EAX conținutul de la adresa et (4 octeți)
lea eax, [v]       ; încarcă în registrul eax adresa (offsetul) variabilei v (4 octeți)
```

Ca generalizare, **folosirea parantezelor pătrate indică întotdeauna accesarea unui operand din memorie**. De exemplu, `mov EAX, [EBX]` semnifică un transfer în EAX a conținutului memoriei a cărei adresă este dată de valoarea lui EBX.

Operanții sunt parametri care definesc valorile ce vor fi prelucrate de instrucțiuni sau de directive. Ei pot fi regiști, constante, etichete, expresii, cuvinte cheie sau alte simboluri. Semnificația operanților depinde de *semnificația instrucțiunii sau directivei*

3.2. EXPRESII

Expresiile sunt evaluate în momentul asamblării (adică, valorile lor sunt determinabile la momentul asamblării, cu excepția acelor părți care desemnează conținuturi de registri și care vor fi determinate la execuție).

3.2.1. Moduri de adresare

Operanzii instrucțiunilor pot fi specificați sub forme numite *moduri de adresare*.

Cele trei tipuri de operanți sunt: *operanți imediați*, *operanți registru* și *operanți în memorie*.

Valoarea operanzilor este calculată în momentul asamblării pentru operanzii imediați, în momentul încărcării programului pentru adresarea directă (adresa FAR) și în momentul execuției pentru operanzii registru și cei adresați indirect.

3.2.1.1. Utilizarea operanzilor imediați

Operanții imediate sunt formați din date numerice constante calculabile la momentul asamblării.

Constantele întregi se specifică prin valori binare, octale, zecimale sau hexazecimale. Adicional, este permisă folosirea caracterului _ (underscore) pentru a separa grupuri de cifre. Baza de numerație poate fi precizată în mai multe moduri:

- Folosind sufixele H sau X pentru hexazecimal, D sau T pentru zecimal, Q sau O pentru octal și B sau Y pentru binar; în aceste cazuri numărul trebuie să înceapă obligatoriu cu o cifră între 0 și 9, pentru a nu exista confuzii între constante și simboluri, de exemplu, OABCH este interpretat ca număr hexazecimal, dar ABCH este interpretat ca simbol
- În stil C, prin prefixare cu 0x sau 0h pentru hexazecimal, 0d sau 0t pentru zecimal, 0o sau 0q pentru octal, respectiv 0b sau 0y pentru binar;

hexa: 6x

Example:

- constanta hexazecimală B2A poate fi exprimată ca 0xb2a, 0xb2A, 0hb2a, 0b12Ah, 0B12AH, etc;
- valoarea zecimală 123 poate fi specificată ca 123, 0d123, 0d0123, 123d, 123D, ...
- 11001000b, 0b11001000, 0y1100_1000, 001100_1000Y reprezintă diferite exprimări ale numărului binar 11001000

Momentul asamblării ← stim valoarea speranței de viață imediate
↳ offset etichete de date și de cod

Momentul încălzirii (Loading time) ← adresa FAU (se generează offset)

Momentul execuției $\leftarrow$ valoarea parametrilor negativi + adresa indirectă

5

Deplasamentele etichetelor de date și de cod reprezintă valori determinabile la momentul asamblării care rămân constante pe tot parcursul execuției programului

`mov eax, et` ; transfer în registrul EAX a adresei (offsetului) asociate etichetei et

va putea fi evaluată la momentul asamblării drept de exemplu

`mov eax, 8` ; distanță de 8 octeți față de începutul segmentului de date

“Constanța” acestor valori derivă din regulile de alocare adoptate de limbajele de programare în general și care statuează că ordinea de alocare în memorie a variabilelor declarate (mai precis distanța față de începutul segmentului de date în care o variabilă este alocată) sau respectiv distanțele salturilor destinație în cazul unor instrucțiuni de tip `goto` sunt valori constante pe parcursul execuției unui program.

Adică: o variabilă odată alocată în cadrul unui segment de memorie nu își va schimba niciodată locul alocării (adică poziția sa față de începutul acelui segment) iar această informație determinabilă la momentul asamblării derivă din ordinea specificării variabilelor la declarare în cadrul textului sursă și din dimensiunea de reprezentare dedusă pe bază informației de tip asociate.

3.2.1.2. Utilizarea operanzilor registru

Modul de invocare/accesare directă - `mov eax, ebx`

Invocare/accesare indirectă - pentru a indica locațiile de memorie - `mov eax, [ebx]`

3.2.1.3. Utilizarea operanzilor din memorie

Operanții din memorie : cu adresare directă și cu adresare indirectă.

Operandul cu adresare directă este o constantă sau un simbol care reprezintă adresa (segment și deplasament) unei instrucțiuni sau a unor date. Acești operanți pot fi etichete (de ex: `jmp et`, `var1 dw 324`, `add eax,[b]`), nume de proceduri (de ex: `call proc1`) sau valoarea contorului de locații (de ex: `b db 5-a`).

Deplasamentul unui operand cu adresare directă este calculat în momentul asamblării (assembly time). Adresa fiecărui operand raportată la structura programului executabil (mai precis stabilirea segmentelor la care se raportează deplasamentele calculate) este calculată în momentul editării de legături (linking time). Adresa fizică efectivă este calculată în momentul încărcării programului pentru execuție (loading time – acest proces final de ajustare a adreselor numindu-se RELOCAREA ADRESELOR = Address Relocation).

Un deplasament utilizat ca operand în cadrul unui program este întotdeauna raportat la un registru de segment. Acest registru poate fi specificat explicit sau, în caz contrar, se asociază de către asamblor în mod implicit un registru de segment. Regulile limbajului de asamblare pentru asocierile implicite sunt:

- CS pentru etichete de cod destinație ale unor salturi (`jmp`, `call`, `ret`, `jz` etc);
- SS în adresări SIB ce folosește EBP sau ESP drept bază (indiferent de index sau scală);
- DS pentru restul accesărilor de date

Specificarea explicită a unui registru de segment se face cu ajutorul operatorului de prefixare segment (notat “:” și care se mai numește, “operatorul de specificare a segmentului”). ES poate fi utilizat numai în specificări explicite (ca de exemplu `ES:[Var]` sau `ES:[ebx+eax*2-a]`) sau în cadrul unor instrucțiuni pe siruri (MOVSB)

`JMP FAR CS:...`

`JMP FAR DS:... or JMP FAR [label2]`

6

Curs 6

Reguli pentru asocierea implicită sunt

- CS pentru etichete de cod destinație ale unor salturi (jmp, call, ret, jz etc);
- SS în adresări SIB ce folosește EBP sau ESP drept bază (indiferent de index sau scală);
- DS pentru restul accesărilor de date

Exemple ilustrând regulile implicite de prefixare ale unui offset cu segmentul aferent

Mov eax, [v] ; mov eax, DWORD PTR DS:[405000]	Mov eax, [ebx+ebp] ; EAX ← ...DS:...
	Mov eax, [ebp+ebx] ;SS:.....
Mov eax, [ebx] ; mov eax, DWORD PTR DS:[ebx]	
Mov eax, [ebp] ; mov eax, DWORD PTR SS:[ebp]	Mov eax, [ebx*2+ebp] ;SS:....
Mov eax, [ebp*2] ; mov eax, DWORD PTR SS:[ebp+ebp]	Mov eax, [ebx*1+ebp] ; ... SS:....
Mov eax, [ebp*3] ; mov eax, DWORD PTR SS:[ebp+ebp*2]	
Mov eax, [ebp*4] ; mov eax, DWORD PTR DS:[ebp*4]	Mov eax, [ebp*1+ebx] ; ...DS:...
Mov eax, [ebx+esp] ; eax ← dword care începe la adresa [SS:esp+ebx]	
Mov eax, [esp + ebx] ; eax ← dword care începe la adresa [SS:esp+ebx]	
Mov eax, [ebx+esp*2] ; syntax error – ESP nu poate fi index !	
Mov eax, [ebx+ebp*2] ; eax ← dword care începe la adresa [DS:ebx + 2*ebp]	
Mov eax, [ebx*1+ebp*1] ; SS... ; Primul gasit cu * e considerat index ! EBP - baza	
Mov eax, [ebp*1+ebx*1] ; DS.... ; Primul gasit cu * e considerat index ! EBX - baza	
Mov eax, [ebp*1+ebx*2] ; ...SS...	

Adresa oricărei variabile alocată într-un program este fixă pe tot parcursul execuției, iar offset-ul său este întotdeauna o constantă determinabilă la momentul asamblării.

7

3.2.1.4. Operanzi cu adresare indirectă

Operanzii cu *adresare indirectă* utilizează regiștri pentru a indica adrese din memorie. Deoarece valorile din regiștri se pot modifica la momentul execuției, adresarea indirectă este indicată pentru a opera în mod dinamic asupra datelor.

Forma generală pentru accesarea indirectă a unui operand de memorie este dată de formula de calcul a offset-ului unui operand:

$$[\text{registru_de_bază}] + [\text{registru_index} * \text{scală}] + [\text{constantă}]$$

Constanta este o expresie a cărei valoare este determinabilă la momentul asamblării. De exemplu, `[ebx + edi + table * 6]` desemnează un operand prin adresare indirectă, unde atât *table* cât și 6 sunt constante.

Operanzii *registru_de_bază* și *registru_index* sunt folosiți de obicei pentru a indica o adresă de memorie referitoare la un tablou. În combinație cu factorul de scalare, mecanismul este suficient de flexibil pentru a permite acces direct la elementele unui tablou de înregistrări, cu condiția ca dimensiunea în octeți a unei înregistrări să fie 1, 2, 4 sau 8. De exemplu, octetul superior al elementului de tip DWORD cu index dat în `ecx`, parte a unui vector de înregistrări al cărui adresă (a vectorului) este în `edx` poate fi încărcat în `dh` prin intermediul instrucțiunii

```
mov dh, [edx + ecx * 4 + 3]
```

Din punct de vedere sintactic, atunci când operandul nu este specificat prin formula completă, lipsind unele dintre componente (de exemplu lipsește "`* scală`"), asamblorul va rezolva ambiguitatea care rezultă printr-un proces de analiză a tuturor formelor echivalente de codificare posibile și alegerea celei mai scurte dintre acestea. Cu alte cuvinte, având

```
push dword [eax + ebx] ; salvează pe stivă dublucuvântul de la adresa eax+ebx
```

asamblorul are libertatea de a considera `eax` drept bază și `ebx` drept index sau invers, `ebx` drept bază și `eax` drept index. Analog, pentru

```
pop DWORD [ecx] ; restaurează vârful stivei în variabila cu adresa dată de ecx
```

asamblorul poate interpreta `ecx` fie ca bază fie ca index. Ce este realmente important de reținut este faptul că toate codificările luate în considerare de către asamblor sunt echivalente iar decizia finală a asamblorului nu are impact asupra funcționalității codului rezultat.

De asemenea, în plus față de rezolvarea unor astfel de ambiguități, asamblorul permite și exprimări non-standard cu condiția ca acestea să fie transformabile într-un final în forma standard de mai sus.

Alte exemple:

```
lea eax, [eax*2] ; încarcă în eax valoarea lui eax*2 (adică, eax devine 2*eax)
```

În acest caz, asamblorul poate decide între codificare de tip bază = `eax + index = eax` și `scală = 1` sau index = `eax` și `scală = 2`.

```
lea eax, [eax*9 + 12] ; eax ia valoarea eax * 9 + 12
```

Deși `scală` nu poate fi 9, asamblorul nu va emite aici un mesaj de eroare. Aceasta deoarece el va observa posibila codificare a adresei drept: bază = `eax + index = eax` cu `scală = 8`, unde de această dată valoarea 8 este corectă pentru `scală`. Evident, instrucțiunea putea fi precizată mai clar sub forma

```
lea eax, [eax + eax * 8 + 12].
```

Să reținem deci că pentru adresarea indirectă, esențială este specificarea între paranteze drepte a cel puțin unuia dintre elementele componente ale formulei de calcul a offsetului.

00401001

```
mov eax, 410000
mov ebx, [eax+2]
```

8

3.2.2. Utilizarea operatorilor

Operatori - pentru combinarea, compararea, modificarea și analiza operanzilor. Unii operatori lucrează cu constante întregi, alții cu valori întregi memorate, iar alții cu ambele tipuri de operanzi.

Este importantă înțelegerea diferenței dintre operatori și instrucțiuni. Operatorii efectuează calcule cu valori constante SCALARE determinabile la momentul asamblării (valori scalare = valori imediate), cu excepția adunării și scăderii unei constante la un/dintr-un pointer (care va furniza întotdeauna "pointer data type") și cu excepția formulei de calcul al offset-ului unui operand (care permite operatorul '+'). Instrucțiunile efectuează calcule cu valori ce pot fi necunoscute până în momentul execuției. Operatorul de adunare (+) efectuează adunarea în momentul asamblării; instrucțiunea ADD efectuează adunarea în timpul execuției.

Operatorii disponibili pentru construcția expresiilor sunt asemănători celor din limbajul C, atât ca sintaxă cât și din punct de vedere semantic. Evaluarea expresiilor numerice se face pe 64 de biți, rezultatele finale fiind ulterior ajustate în conformitate cu dimensiunea de reprezentare disponibilă în contextul de utilizare al expresiei.

În tabelul de mai jos sunt prezentați în ordinea priorității operatorii ce pot fi folosiți în cadrul expresiilor limbajului de asamblare x86 (NASM !!).

Prioritate	Operator	Tip	Rezultat
7	-	Unar, prefixat	Complement față de 2 (negare): $-X = 0 - X$
7	+	Unar, prefixat	Fără efect (oferit pentru simetrie cu „-“): $+X = X$
7	~	Unar, prefixat	Complement față de 1: $\text{mov al, } -0 \Rightarrow \text{mov AL, } 0xFF$
7	!	Unar, prefixat	Negare logică: $!X = 0$ când $X \neq 0$, altfel 1
6	*	Binar, infix	Înmulțire: $1 * 2 * 3 = 6$
6	/	Binar, infix	Câtul împărțirii fără semn: $24 / 4 / 2 = 3$ ($-24 / 4 / 2 = 0FDh$)
6	//	Binar, infix	Câtul împărțirii cu semn: $-24 // 4 // 2 = -3$ ($-24 / 4 / 2 \neq -3!$)
6	%	Binar, infix	Restul împărțirii fără semn: $123 \% 100 \% 5 = 3$
6	%%	Binar, infix	Restul împărțirii cu semn: $-123 \% 100 \% 5 = -3$
5	+	Binar, infix	Însumare: $1 + 2 = 3$
5	-	Binar, infix	Scădere: $1 - 2 = -1$
4	<<	Binar, infix	Deplasare pe biți către stânga: $1 << 4 = 16$
4	>>	Binar, infix	Deplasare pe biți la dreapta: $0xFE >> 4 = 0x0F$
3	&	Binar, infix	ȘI: $0xF00F \& 0xFF0F = 0x000F$
2	^	Binar, infix	SAU exclusiv: $0xFF0F \wedge 0xFF0F = 0x0000$
1		Binar, infix	SAU: $1 2 = 3$

Operatorul de indexare are o utilizare largă în specificarea operanzilor din memorie adresați indirect. Paragraful 3.2.1 a clarificat rolul operatorului [] în adresarea indirectă.

Acest tabel este de o importanță covârșitoare în anumite situații și e bine să îl avem „la îndemână”. Iată de ce:

$5[6+7\&8] = (5[6])+(7\&8) = 7+0 = 7$?? NU !!!!! datorită precedenței operatorilor avem;

$5[6+7\&8] = 5[(6+7)\&8] = 5[13\&8] = 5[8] = 13 = 0Dh$!!!

3.2.2.3. Operatori de deplasare de biți

expresie >> cu_cât și expresie << cu_cât

mov ah, 01110111b << 3; desemnează valoarea 10111000b

add bh, 01110111b >> 3; desemnează valoarea 00001110b

AX 00000111 10111000 !!!!
BX 00000000 00001110b

! , ~ , + , -
/ , / , % , % , %
+ , - (binar)
&
^ sau exclusiv
| sau

9

3.2.2.4. Operatori logici pe biți

Operatorii pe biți efectuează operații logice la nivelul fiecărui bit al operandului (operandilor) unei expresii. Expresiile au ca rezultat valori constante.

OPERATOR	SINTAXA	SEMNIFICAȚIE
~	~ expresie	complementare biți
&	expr1 & expr2	ȘI bit cu bit
	expr1 expr2	SAU bit cu bit
^	expr1 ^ expr2	SAU exclusiv bit cu bit

! Instructioni
aproximative
not
and
or
xor

Exemple (presupunem că expresia se reprezintă pe un octet):

~ 11110000b ; desemnează valoarea 00001111b ; ~0f0h = ~...
 01010101b & 11110000b ; are ca rezultat valoarea 01010000b
 01010101b | 11110000b ; are ca rezultat valoarea 11110101b
 01010101b ^ 11110000b ; are ca rezultat valoarea 10100101b

! – negare logică (similar cu limbajul C) ; !0 = 1 ; !(orice diferit de zero) = 0

3.2.2.6. Operatorul de specificare a segmentului

Operatorul de specificare a segmentului (:) comandă calcularea adresei FAR a unei variabile sau etichete în funcție de un anumit segment. Sintaxa este:

segment:expresie

[ss: ebx+4] ; deplasamentul e relativ la SS // [es: 082h] ; deplasamentul e relativ la ES
 10h:var ; segmentul este indicat de selectorul 10h, iar offsetul este valoarea etichetei var.

3.2.2.7. Operatori de tip

Specifică tipurile unor expresii și a unor operanzi păstrați în memorie. Sintaxa pentru aceștia este

tip expresie

unde specificatorul de tip este unul dintre cuvintele cheie **BYTE**, **WORD**, **DWORD**, **QWORD**.

Această construcție sintactică forțează ca *expresie* să fie tratată ca având dimensiunea de reprezentare *tip*, fără însă a-i modifica definitiv (distructiv) valoarea în sensul precizat de conversia dorită. De aceea, aceștia sunt considerați operatori de conversie (temporară) nedistructivă. Pentru operanzii păstrați în memorie, *tip* poate fi **BYTE**, **WORD**, **DWORD**, **QWORD** având dimensiunile de reprezentare 1, 2, 4, 8 octeți. Pentru etichetele de cod el poate fi **NEAR** (adresă pe 4 octeți) sau **FAR** (adresă pe 6 octeți). Expresia **byte [A]** va indica doar primul octet de la adresa indicată de A. Analog, **dword [A]** indică dublucuvântul ce începe la adresa A.

Specificatorii **BYTE** / **WORD** / **DWORD** / **QWORD** au întotdeauna doar rol de a clarifica o ambiguitate (inclusiv când este vorba despre o variabilă de memorie, faptul de a preciza **mov BYTE [v], 0** sau **mov WORD [v], 0** este tot o clarificare a ambiguității, cum **nasm** nu asociază faptul că v este **byte** / **word** / **dword**).

mov [v], 0 ; syntax error – operation size not specified

Specificatorul QWORD nu intervine niciodată explicit în cod pe 32 de biți.

Exemple unde e necesar un specificator de dimensiune al operanzilor:

mov [mem], 12 ; (:)div [mem] ; (:)mul [mem]

push [mem] ; pop [mem]

- **push 15** - aici este o inconsistență în **NASM**, asamblorul nu va emite eroare/warning ci va face
- **push DWORD 15**

10

Exemple de operanzi IMPLICITI efectiv pe 64 biți (în cod pe 32):

- **mul dword [v]** ; înmulțește eax cu dword-ul de la adresa v și depune în EDX:EAX rezultatul
- **div dword [v]** ; împărțire EDX:EAX la v

3.3. DIRECTIVE

Directivele indică modul în care sunt generate codul și datele în momentul asamblării.

3.3.1.1. Directiva SEGMENT

Directiva SEGMENT permite direcționarea octeților de cod sau date emiși de către un asamblor înspre segmentul precizat, segment care poartă un nume și are asociate diverse caracteristici.

SEGMENT *nume* [*tip*] [*ALIGN=aliniere*] [*combinare*] [*utilizare*] [*CLASS=clasă*]

Numelui segmentului i se asociază ca valoare adresa de segment (32 biți) corespunzătoare poziției segmentului în memorie în faza de execuție. În acest sens, asamblorul NASM pune la dispoziție și simbolul special \$5 care este echivalent cu adresa segmentului curent, acesta având însă avantajul că poate fi utilizat în orice context, fără a fi necesar să fie cunoscut numele segmentului în care ne aflăm.

Cu excepția numelui, toate celelalte câmpuri sunt opționale atât din punct de vedere a prezenței cât și a ordinii în care sunt specificate.

Argumentele opționale *tip*, *aliniere*, *combinare*, *utilizare* și *'clasa'* dau editorului de legături și asamblorului indicații referitoare la modul de încărcare și atributele segmentelor.

Tip permite selectarea unui model de folosire al segmentului, având la dispoziție următoarele opțiuni:

- **code** (sau **text**) - segmentul va conține cod, conținutul nu poate fi scris dar se poate citi sau executa
- **data** (sau **bss**) - segment de date permițând citire și scriere însă nu și execuție (valoare implicită)
- **rdata** - segment din care se poate doar citi, menit a conține definiții de date constante

Argumentul opțional *aliniere* specifică multiplul numărului de octeți la care trebuie să înceapă segmentul respectiv. Alinierele acceptate sunt puteri a lui 2, între 1 și 4096.

Dacă argumentul *aliniere* lipsește, atunci se consideră implicit că este vorba despre o aliniere **ALIGN=1**, adică segmentul poate începe la orice adresă.

Argumentul opțional *combinare* controlează modul în care segmente cu același nume din cadrul altor module vor fi combinate cu segmentul în cauză la momentul editării de legături. Valorile posibile sunt:

- **PUBLIC** - indică editorului de legături să concateneze acest segment cu alte eventuale segmente cu același nume, obținându-se un unic segment a cărei lungime este suma lungimilor segmentelor componente.
- **COMMON** - specifică faptul că începutul acestui segment trebuie să se suprapună peste începutul tuturor segmentelor ce au același nume. Se obține un segment având dimensiunea egală cu cea a celui mai mare segment având același nume.
- **PRIVATE** - indică editorului de legături că acest segment nu este permis a fi combinat cu altele care poartă același nume.
- **STACK** - segmentele cu același nume vor fi concatenate. În faza de execuție segmentul rezultat va fi segmentul stivă.

Implicit, dacă nu se specifică o metodă de combinare, orice segment este considerat **PUBLIC**.

Argumentul *utilizare* permite optarea pentru altă dimensiune de cuvânt decât cea de 16 biți, care este implicită în lipsa precizării acestui argument.

Argumentul *'clasa'* are rolul de a permite stabilirea ordinii în care editorul de legături plasează segmentele în memorie. Toate segmentele având aceeași clasă vor fi plasate într-un bloc contiguu de memorie indiferent de

11

ordinea lor în cadrul codului sursă. Nu există o valoare implicită de inițializare pentru acest argument, el fiind nedefinit în lipsa specificării, ducând în consecință la evitarea concatenării într-un bloc continuu a tuturor segmentelor definite astfel.

```
segment code use32 class=CODE      segment data use32 class=DATA
```

3.3.2. Directive pentru definirea datelor

definire date = declarare (specificarea atributelor) + alocare (rezervarea sp. de memorie necesar).

UNICA !!! **NU e unica !!!** **UNICA !!!**

În C – 17 module (fișiere separate) ; A1- variabilă globală (**int A1**) + 16 declarații de data **extern int A1**;
LINKER-ul este responsabil pt verificarea DEPENDENTELOR dintre module !

Structura unei Variable = { [nume] , set_de_atribute , [adresa_referință , valoare] }

Variabilele dinamice NU AU NUME !!!!

P=new(...); p=malloc(...); ...free... (Diferența between POINTER and DYNAMIC variables !!!!)

(Nume, set_de_atribute) = parametrii formali ai funcțiilor și procedurilor !!!

Set_de_atribute = (TD, Domeniu_de_vizibilitate (scope), durata de viață (lifetime/extent), clasa de memorie)
Memory class (in C) = (auto, register, static, extern)

tipul de dată = dimensiunea de reprezentare – octet, cuvânt, dublucuvânt, quadword

Forma generală a unei linii sursă în cazul unei declarații de date este:

[nume] tip_data lista_expresii [:comentariu]

sau [nume] tip_alocare factor [:comentariu]

sau [nume] TIMES factor tip_data lista_expresii [:comentariu]

unde *nume* este o etichetă prin care va fi referită data. Tipul rezultă din tipul datei (dimensiunea de reprezentare) iar valoarea este adresa la care se va găsi în memorie primul octet rezervat pentru data etichetată cu numele respectiv.

factor este un număr care indică de câte ori se repetă lista de expresii care urmează în paranteză.

Tip_data este o directivă de definire a datelor, una din următoarele:

DB - date de tip octet (BYTE)

DW - date de tip cuvânt (WORD)

DD - date de tip dublucuvânt (DWORD)

DQ - date de tip 8 octeți (QWORD - 64 biți)

DT - date de tip 10 octeți (TWORD - utilizate pentru memorarea constantelor BCD sau constantelor reale de precizie extinsă)

De exemplu, secvența următoare definește și inițializează cinci variabile de memorie:

```
segment data
var1 DB 'd' ;1 octet
.a DW 101b ;2 octeți
var2 DD 2bfb ;4 octeți
.a DQ 307o ;8 octeți (1 quadword)
.b DT 100 ;10 octeți
```

12

NASM does not support the MASM/TASM syntax of reserving uninitialised space by writing `DW ?` or similar things: this is what it does instead. The operand to a `RESB`-type pseudo-instruction is a **critical expression** (toți operanzii care intervin în calcul trebuie să fie cunoscuți în momentul în care expresia este întâlnită). Ex:

```
buffer:    resb  64    ; reserve 64 bytes
wordvar:   resw   1     ; reserve a word
realarray  resq   10    ; array of ten reals
```

Directiva `TIMES` permite asamblarea repetată a unei instrucțiuni sau definiții de dată:

`TIMES factor tip_data expresie`

De exemplu

`Tabchar TIMES 80 DB 'a'`

crează un tablou de 80 de octeți inițializați fiecare cu codul ASCII al caracterului 'a'.

`matrice10x10 times 10*10 dd 0`

va furniza 100 de dublucuvinte dispuse continuu în memorie începând de la adresa asociată etichetei `matrice10x10`.

`TIMES` can also be applied to instructions:

`TIMES factor instrucțiune`

`TIMES 32 add eax, edx` ; having as effect $EAX = EAX + 32 * EDX$

3.3.3. Directiva `EQU`

Directiva `EQU` permite atribuirea, în faza de asamblare, unei valori numerice sau șir de caractere unei etichete fără alocarea de spațiu de memorie sau generare de octeți. Sintaxa directivei `EQU` este

`nume EQU expresie`

Exemple:

```
END_OF_DATA EQU '!'
BUFFER_SIZE EQU 1000h
INDEX_START EQU (1000/4 + 2)
VAR_CICLARE EQU i
```

Prin utilizarea de astfel de echivalări textul sursă poate deveni mai lizibil. Se observă asemănarea etichetelor echivalate prin directiva `EQU` cu constantele din limbajele de programare de nivel înalt.

Expresia pentru echivalarea unei etichete definite prin directiva `EQU` poate conține la rândul ei etichete definite prin `EQU`:

```
TABLE_OFFSET EQU 1000h
INDEX_START EQU (TABLE_OFFSET + 2)
DICTIONAR_STAR EQU (TABLE_OFFSET + 100h)
```

Curs 7

Operații și operatori pe biti

Operatorii și instrucțiunile nu sunt la fel:

mov AH, 0111011b $\ll$ 3 ; AH: 10111000b

VS

mov AH, 0111011b

shl AH, 3

0 - operatorul și bit cu bit

AND - instrucțiune

! $X \text{ AND } 0 = 0 \rightarrow$ forțarea valorii anumitor biți la valoarea 0

$X \text{ AND } 1 = X$

$X \text{ AND } X = X$

$X \text{ AND } \bar{X} = 0$

1 - operatorul SAU bit cu bit

OR - instrucțiune

$X \text{ OR } 0 = X$

! $X \text{ OR } 1 = 1 \rightarrow$ forțarea valorii anumitor biți la valoarea 1

$X \text{ OR } X = X$

$X \text{ OR } \bar{X} = 1$

^ - operatorul SAU EXCLUSIV bit cu bit

XOR - instrucțiune

$X \text{ XOR } 0 = X$

$X \text{ XOR } 1 = \bar{X} \rightarrow$ operație utilă pt complementarea valorii anumitor biți

$X \text{ XOR } X = 0$

$X \text{ XOR } \bar{X} = 1$

! $X \text{ XOR } AH, AH ; AH = 0$

Utilizarea operatorilor ! și ~

! - negarea logică $!x = \begin{cases} 0, & x \neq 0 \\ 1, & \text{altfel} \end{cases}$

~ - complementul față de 1: $\text{mov AL}, \sim 0 \Rightarrow \text{mov AL}, 0FFh$

Exemple

a d? $1\&/21/\dots$ b d? $-5/\dots$ mov eax, ![a]; expression **syntax error** pt că [a] nu este o constantă determinabilă la momentul asamblării

mov eax, [!a]; ! can only be applied to scalar values, a-pointer

mov eax, !a; _____ || _____

mov eax, !(a+7); _____ || _____

mov eax, !(b-a); a, b pointers, dar b-a scalar

mov eax, ![a+7]; expression **syntax error**

mov eax, !7; EAX=0

mov ah, ~7; 7=00000111b deci ~7=11111000b = F8h

mov eax, !0; eax=1

mov eax, !ebx; **syntax error** → val nedeterminabilă la momentul asamblării

aa equ 2 { OK

mov ah, !aa

mov ah, 17^ (~17); AH=11111111b = 0FFh

mov ax, value ^ ~value; AX=0FFFFh

Operatori de tip și tipuri de date asociate operandilor

Operatorii efectuează calcule cu valori constante **scalare** determinabile la momentul asamblării (valori scalare = valori imediate) cu excepția adunării și scăderii unei constante la un/dintr-un pointer (care va furniza întotdeauna „pointer data type”) cu excepția formului de calcul al offset-ului unui operand (care permite operatorul +)

Specificatorii BYTE/WORD/DWORD/QWORD au rolul de a clarifica o ambiguitate

v d?

a d?

b d?

push v, ok, se va pune offset-ul pe stivă pe 32 de biți

push [v]; **syntax error (ambiguitate)**: operator size not specified → push pe stivă acceptă operandi și de 32 și de 16 biți

push dword [v] ✓

push word [v] ✓

mov eax, [v]; ok ✓ EAX = dword ptr [v]; în Ollydbg „mov eax, dword ptr [DS:v]”

push [eax]; **syntax error: Operation size not specified** !push **byte** [eax]; **syntax error**

push word [eax]; ok ✓

push 15; push DWORD 15

pop [v] ; operation size not specified (a pop from the stack accepts both 16 and 32 bits value)

pop word/dword [v] ; ok ✓

pop v ; sintaxa este pop destinație ; destinație must be L-value

↳ R-value ; acest pop este similar cu operatie cu 2 = 3
(invalid combination of opcode and operands)

push op-sursă
pop op-dest

pop dword b ; syntax error

pop [eax] ; operand size not specified

pop (d)word [eax] ; ok ✓

pop 15 ; 15 is NOT a L-value syntax error

pop [15] ; syntax error → operand size not specified

pop dword [15] ; syntactic ok dar cel mai probabil run-time error pt că [DS:15] va provoca access violation

mov [v], 0 ; syntax error - operand size not specified

mov byte [v], 0 ; ok ✓

mov [v], byte 0 ; ok

div [v] ; operand size ?

div word [v] ; ok ✓

imul [v+2] ; operand size ?

imul word [v+2] ; DX:AX = AX * word de la adresa v+2

a d?

b d?

mov a, b ; syntax error pt că NU este L-value, ci R-value fiind un offset determinabil ca și constantă la momentul asamblării

mov [a], b ; syntax error - operand size not specified ! bcs an offset is either a 16 bits value or a 32 bits value
never a 8 bits value (the same effect as mov ah, v)

mov word [a], b ; ok ✓ → 2 octeți inferenți din valoarea offset-ului lui b!

mov dword [a], b ; full offset 32 bits

mov a, [b] ; invalid combination of opcode and operands (a = R-value)

mov [a], [b] ; invalid combination of opcode and operands (NU putem avea 2 operanzi simultani din memorie)

mul v ; syntax error MUL reg/mem not offset

mul word v ; — || —

mul [v] ; op. size not specified

mul dword [v] ; ok ✓

mul eax; ok ✓
mul [eax]; operand size not specified
mul byte [eax]; ok ✓
mul 15 ; invalid comb of opcode and operands - MUL reg/mem, not constant
pop byte [v] ; invalid comb of opcode and operands
pop ~~q~~word [v] ; instructions not supported in 32 bit mode

Clasificarea erorilor în informatică

- **Eroare de sintaxă** – ea este diagnosticată de asamblor/compiler !
(eroare de asamblare)
- **Run-time error (eroare la execuție)** – programul “crapă” – programul se oprește
= program crash !!
- **Eroare logică** = programul funcționează până la capăt sau rămâne blocat în
ciclu infinit, însă GREȘIT dpdv LOGIC obținându-se cu totul alte rezultate decât
cele așteptate...
- **Fatal: Linking Error !!!** (de ex în cazul unei definiții duble de variabilă... 17 module
și o variabilă trebuie să fie DEFINITĂ DOAR într-un singur modul ! Dacă ea este
definită în 2 sau mai multe module se va obține Fatal: Linking Error !!! –
Duplicate definition for symbol A1 !!!)

Atunci DE CE mai asociem în definiție o directivă de tip de dată ??? Pt a INDICA ASAMBLORULUI CUM să populeze cu date/inițializeze zona de memorie respectivă !!! (fie ca o secvență de bytes, fie ca una de words, fie ca una de doublewords !) Directivele de date se referă la modalitatea de inițializare concretă a unei zone de memorie și nu asociază atributul de tip de dată cu un simbol !

Deci: directiva de definire a unei date NU este un mecanism de asociere de tip de dată pentru o variabilă, ci doar un mecanism de inițializare a zonei de memorie alocate variabilei cu valorile dorite !!!

Știute, dar bine de a fi reamintite:

The *name of a variable* is *associated* in assembly language *with its offset relative to the segment* in which its *definition appears*. The *offsets of the variables defined in a program are always constant* values, determinable at assembly/compiling time.

Assembly language and *C* are *value oriented languages*, meaning that everything is reduced in the end to a numeric value, this being a low level feature.

In a *high-level programming language*, the *programmer can access the memory only* by using *variable names*, in contrast, in *assembly language*, the *memory is/can/must be accessed ONLY* by using the *offset computation formula* ("formula de la doua noaptea") where *pointer arithmetic* is also used (pointer arithmetic is also used in C !).

mov ax, [ebx] – the source operand *doesn't* have an *associated data type* (it represents only a start of a memory area) and because of that, in the case of our MOV instruction the *destination operand* is the one that *decides the data type of the transfer* (a word in this case), and the transfer will be made accordingly to the little endian representation.

The steps followed by a program from source code to run-time:

- Syntactic checking (done by assembler/compiler/interpreter)
- OBJ files are generated by the assembler/compiler
- Linking phase (performed by a LINKER = a tool provided by the OS, which checks the possible DEPENDENCIES between this OBJ files/modules); The result □ .EXE file !!!
- You (the user) are activating your exe file (by clicking or enter-ing...)
- The LOADER of the OS is looking for the required RAM memory space for your EXE file. When finding it, it loads the EXE file AND performs ADDRESS RELOCATION !!!!
- In the end the loader gives control to the processor by specifying THE PROGRAM's ENTRY POINT (ex: the start label) !!! The run-time phase begins NOW...

Mark Zbirkowski – semnătura EXE = 'MZ'

Tipuri de date asociate operanzilor**Directivile de definire a datelor în NASM NU sunt mecanisme de definire a tipurilor de date !!**

a db 17,19

b dw 1234h ; 34h 12h c dd....

Rolele directivelor de definire a datelor NU este în NASM de a preciza tipul de dată al variabilelor definite, ci DOAR de a genera octeții corespunzătoare acelor zone de memorie pe care le ocupă în conformitate cu directiva specifică aleasă și respectând ordinea de plasare de tip little-endian !!!

Deci a NU este de tip byte – ci doar un offset/deplasament și atât... un simbol desemnând începutul unei zone de memorie FĂRĂ VREUN TIP ASOCIAT !

Iar b NU este de tip word – ci doar un offset/deplasament și atât ... un simbol desemnând începutul unei zone de memorie FĂRĂ VREUN TIP ASOCIAT !

Și nici c NU este de tip doubleword – ci doar un offset/deplasament și atât ... un simbol desemnând începutul unei zone de memorie FĂRĂ VREUN TIP ASOCIAT !

Laborator 4

AND - pune rezultatul în primul operand

ex) `and eax, 0Fh` → zeroizarea tuturor bitilor mai puțin ultimii 4

A	B	A and B	A or B	A xor B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

OR - pune rez în primul operand

ex) `or eax, 0Fh`; $eax \leftarrow$ ultimii 4 biti vor fi 1, restul neschimbate

XOR - sunt A și B NU egale?

dacă A și B egale = 0

dacă $A \neq B = 1$

ex) `xor ax, ax`; $ax \leftarrow 0$ pt că $ax = ax$

! se folosește pt zeroizarea registrilor

TEST - operația AND fără să pună rez în registru/operand

ex) `test AL, 0Fh` → testează dacă m e par sau impar

↳ dacă

→ afișează ZF, SF, PF

NOT - inversarea fiecărui bit din operand

ex) `not byte [var]` ptr [var];

SHL - bitii stocați în destinație x deplasare m de poz (%32)

spre stânga. Bitul (sau bitii) cel mai din dreapta se completează cu 0. Ultimul bit care iese în stânga se păstrează în CF

`shl <reg>/<imm>, <op>/CL`

ex) `mov al, 00110011b`

`mov cl, 2`

`shl al, cl`, $al = 11001100b$, $CF = 0$

SHR → SHL dar în dreapta, bitii din stânga vor fi 0

ex) `mov al, 01011110b`
`mov cl, 2`
`shr al, cl ; al ← 00010111b, CF = 1`

SAL = SHL

SAR ≈ SHR, dar ea bitii al mai din stânga se completează cu bitul de semn

ROL - bitii deosebiți în destinație se releasează în poz spre stânga.

Odată un bit ieșit în stânga el se adaugă automat în partea dreaptă a destinației. Ultimul bit rotit se păstrează în CF

ex) `mov al, 00110011b`
`mov cl, 2`
`rol al, cl ; al = 11001100b, CF = 0`

ROR - ROL, dar spre dreapta

ex) ~~`mov al, 00110011b`~~
~~`mov cl, 2`~~
~~`ror al, cl ; al = 11001100b, CF = 0`~~
`mov al, 00111101b`
`mov cl, 2`
`ror al, cl ; al = 10001111b, CF = 1`

RCL - bitii deosebiți în destinație se releasează în poz spre stânga. Odată un bit ieșit în stânga el se păstrează în CF. Valoarea anterioară din CF se adaugă automat în partea dreaptă a destinației

ex) `stc ; CF = 1` + **RCL** ≈ RCL dar la dreapta
`mov al, 00110011b`
`mov cl, 2`
`rcl al, cl ; al = 11001110b, CF = 0`

CAPITOLUL 4

Instrucțiuni ale limbajului
de asamblare

Forma generală a unui program NASM + sunt exemple:

global start ; solicităm asamblorului să confere vizibilitate globală simbolului denumit
start (diferența start va fi punctul de intrare în program)

extern ExitProcess, printf ; informăm asamblorul că simbolurile -1- au
proveniență străină, evităm erorile

import ExitProcess kernel32.dll ; precizăm care sunt bibliotecile externe care definesc
cele două simboluri

bits 32 ; solicităm asamblarea pt un procesor x86 (pe 32 biți)

segment data use32 class=DATA
string: db "Salut din ASM!", 0

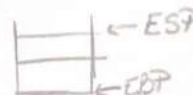
segment code use32 class=CODE
start:

; apel printf("Salut din ASM!")
push dword string
call [printf]

; apel ExitProcess(0), 0 reprezentând "execuție cu succes"
push dword 0
call [ExitProcess]

mov al, 300 ; AL ← 44
warning byte value
exceeds bounds

Instrucțiuni de transfer general



mov d,s	$\langle d \rangle \leftarrow \langle s \rangle$ b-b dw-dw w-w
push s	$ESP = ESP - 4$ și depune $\langle s \rangle$ în stivă (s-dword)
pop d	extrage elementul din stivă și îl depune în (d-dword) $ESP = ESP + 4$
xchg d,s	$\langle d \rangle \leftrightarrow \langle s \rangle$, interschimbă, L-values
[reg-segm] XLAT	$AL \leftarrow \langle DS:[EBX + AL] \rangle$ sau $AL \leftarrow \langle reg-segment:[EBX + AL] \rangle$
cmovec d,s	$\langle d \rangle \leftarrow \langle s \rangle$ dacă cc (cod condiție) este adevărat
pusha / pushad	depune pe stivă EAX, ECX, EDI, ESI, ESP, EBP, EBP, ESI și EDI
popa / popad	extrage EDI, ESI, EBP, ESP, EBP, EBP, ESI și EAX de pe stivă
pushf / pushfd	depune EFlags pe stivă
popf / popfd	extrage valoarea stivei și îl depune în EFlags
setcc d	$\langle d \rangle \leftarrow 1$ dacă cc este adevărat, altfel $\langle d \rangle \leftarrow 0$

→ dacă unul dintre cei 6 regiștri de segment (DS, CS, ES, SS, FS, GS) este de adresa s trebuie să fie unul dintre cei 8 regiștri generali de 16 biți ai UE sau o variabilă de memorie

openamzu : duurd !

Stiva crește de la adrese mai mari spre adrese mai mici, din 4 în 4 octeți,
ESP punând întotdeauna spre dublu cuvântul din vârful stivei.

Structuri echivalente:

push eax $\Leftrightarrow$ sub esp, 4
mov [esp], eax

```
pop eax    =>  mov eax, [esp]
               add esp, 4
```

! stiva este admisă dacă
conține valori cu dimensiuni
mulțipli de 4 (dword & quad)

exemplu

push esp sau pop dword [esp]

Ordinea operațiilor:

- a) se evaluează operandul instrucțiunii (ESP sau respectiv dword [ESP])
- b) se actualizează conținutul ESP ($ESP := ESP - 4$ pt push și $ESP := ESP + 4$ pt pop)
- c) se efectuează atribuirea implicată de efectul instrucțiunii

Presupunem că situația inițială: $ESP = 00A9FF74$

push ESP $\Rightarrow ESP = 00A9FF70$ și
vârf stivă: $00A9FF74$

Presupunem că situația inițială: $ESP = 00A9FF74$ și val din vîl stack: $7741FA29$



XCHG operand1, operand2

↳ interschimbarea conținutului a doi operanzi de aceeași dimensiune, cel puțin unul trebuie să fie registru

[reg-segment] XLAT

= "traduce" octetul din AL într-un alt octet, utilizând în acest scop o tabelă de corepondență creată de utilizator, numită tabelă de traducere

↓
adresa directă a unui șir de octeți

instrucțiunea XLAT permite la intrare adresa fan a tabelui de traducere prin furnikată prin una din modurile:

- DS:EBX (implicit, dacă lipsește precizarea registrului segment)
- registru-segment:EBX, dacă registrul segment este precizat explicit

⇒ efectul instrucțiunii XLAT este înlocuirea octetului din AL cu octetul din tabelă ce are numărul de ordine valoarea din AL (primul are index 0)

exemplu

```
mov ebx, Tabelai
mov al, 6
ES xlat ; AL ← <ES:[EBX+6]>
```

după ce conținutul celui de-a 7-a
locuș de memorie (de index 6) din
Tabela pm AL

exemplu

```
segment data use 32
...
TabHexa db '0A23456789ABCDE', 0
...
segment code use 32
mov ebx, TabHexa
...
mov al, numari
xlat ; AL ← <DS [EBX+1AL]>
```

(3)

Instrucțiunea de transfer al datelor LEA

LEA reg-general, continutul unui operand din memorie $\Leftrightarrow$ reg-general $\leftarrow$ offset (mem)

LEA (Load Effective Address) transferă offset-ul operandului din mem în registru

$\text{lea eax, [v]} \Leftrightarrow \text{mov eax, v}$

avantaj: operandul sursă poate fi o expresie de adusare

ex: $\text{lea eax, [ebx + v - 6]}$ cu efect $\text{mov eax, ebx + v - 6}$ dar nu există o singură instrucțiune mov echivalentă pt ea nu ar fi corectă sintactic

exemplu: Înmulțirea unui nr cu 10

mov eax, [numen] ; $\text{eax} \leftarrow \text{val variabilei numen}$

$\text{lea eax, [eax * 2]}$; $\text{eax} \leftarrow \text{numen} * 2$

$\text{lea eax, [eax * 4 + eax]}$; $\text{eax} \leftarrow (\text{eax} * 4) + \text{eax} = \text{eax} * 5 = (\text{numen}) * 2 * 5$

Prin utilizarea directă a valorilor deplasamentelor se rezultă în urma calculului de adresă (în contrast cu folosirea memoriei indicate de către acestea), LEA se evidențiază prin versatilitate și eficiență sporite:

→ versatilitate prin combinarea unei înmulțiri cu adunarea de registre și/sau valori constante

→ eficiență ridicată datorată executiei întregului calcul într-o singură instrucțiune fără a ocupa circuitele ALU care nu sînt astfel disponibile pt alte operații (timp în care calculul de adresă este efectuat de către circuite specializate, separate, ale BIU)

Instrucțiuni asupra flag-urilor

LAHF (Load register AH from Flags) - copierează SF, ZF, AF, OF și CF din registrul de flaguri în bitii 7, 6, 4, 3 și 0 ai AH. Conținutul bitilor 5, 3, 1 nedif.

SAHF (Store register AH into Flags) - transferă bitii 7, 6, 4, 3, 0 din AH în flag-uri

PUSHF / POPF - transferă / extrage conținutul flag-urilor (dei) pe stivă

Instrucțiuni de setare a flag-urilor

CLD - clear direction flag $\rightarrow \text{DF} = 0$

STD - set direction flag $\rightarrow \text{DF} = 1$

CLE - clear carry flag $\rightarrow \text{CF} = 0$

CMC - complement carry flag $\rightarrow \text{CF} = \neg \text{CF}$

STC - set carry flag $\rightarrow \text{CF} = 1$

STI, CLI $\rightarrow$ programarea pe 16 bit